

Session 5 · Agents, Governance, Your Capstone

Until now you wrote every plan. Now the model plans its own steps, within limits you define — and your finished tool goes public on GitHub.

Plan: concepts 10' · demo 18' · lab 22' · capstones 20' · close 5'

By the end of Session 5 you can

1. **Explain** what separates an agent from a workflow, and when each wins
2. **Read** an agent loop and point at every governance lever in the code
3. **Run** a governed agent on your own company pair, and pass its human gate
4. **Publish** your finance tool on GitHub, with a README that claims only what the tool does
5. **Present** your AI workflow in five minutes, with a trust story

What we are doing, and why

- **An agent is a model granted autonomy:** it chooses which tool to call next, in a loop, until it judges the goal met — or reaches limits you set.
- **Autonomy is granted like a mandate:** explicit rules, hard limits, a human signature at the end. You write all three today, in code.
- **The whole mechanism is about forty lines,** and you will read every one of them before granting the model any autonomy.
- **The industry frameworks come last:** PydanticAI runs the same discipline with your own tool, and a mapping table places every lever in LangGraph, whose workflow half you built in Session 4.

The storyline of this session

the agent's anatomy



Apple vs Sony



the currency check, in code



your rules govern the run



the same agent in PydanticAI



publish on GitHub

forty lines, every governance lever visible

the task – and Sony files in Japanese yen

your tool detects mixed units and raises a warning

the agent plans, fetches, compares – governed

your tool, unchanged, in an industry framework

the finished tool, public under your name

Workflow vs agent

	Workflow (Session 4)	Agent (now)
The plan	fixed, written by you	chosen by the model , step by step
Tools	your code calls them	the model requests, your code executes
Stops when	the script ends	when it decides, or at the limits you set
Use for	repeatable work	paths that depend on the data

The agent mechanism is a loop

```
while steps < MAX_STEPS:
    response = claude(messages, tools=TOOLS)
    if response.wants_tool:
        result = run_tool(...)          # your code executes
        messages += [response, result]  # the conversation is the memory
    else:
        break
```

About forty lines. Once you have read them, you can evaluate any vendor's agent claims from first principles — the notebook then runs the same discipline in **PydanticAI** and maps every lever to **LangGraph**.

Governance: six levers, all in code

Lever	Failure it prevents
MAX_STEPS	the agent looping indefinitely
TOKEN_BUDGET	unbounded spend
Tool whitelist	doing anything you gave no tool for
Forced output schema	an essay instead of a decision
Human gate	anything saved without explicit approval
Audit trail	decisions that cannot be reconstructed afterwards

The demonstration: a currency trap

Task: compare Apple with Sony — its oldest consumer-electronics rival.

naive agent:	JPY 13tn vs USD 416bn → "Sony is 30x bigger"	(wrong)
our agent:	tool detects mixed units → warning → compares growth and margins only	(correct)

- **Defense in depth:** a rule in the prompt *and* a check in the tool
- You build both protections yourself, then run your own pair

The libraries in this session

Library	Role here	Standing
<code>pydantic</code>	the typed recommendation	the industry standard for validation
PydanticAI	the agent loop, packaged — runs live, using your own tool	a leading agent framework, from the Pydantic team
LangGraph	agents as graphs (you built its workflow half in Session 4)	the other leading agent framework

How the labs work

Every exercise sits between two markers. **You fill the gaps. Nothing else changes.**

```
### START CODE HERE ###  
"3. ALWAYS check the [WHICH FIELD?] field before comparing figures."  
### END CODE HERE ###
```

- **None** → replace with the correct column, value or variable
- **[QUESTION IN CAPITALS]** → replace with the text the bracket asks for
- **Everything else is given.** Do not rewrite it.
- Then run the  **check cell** directly below. Green means correct: continue.

Stuck for two minutes? Select the lines, press **Option+K** (**Alt+K** on Windows), and ask the ***** panel.

Your lab, then your five minutes

Lab (22'): build the currency tool + the agent's rules → run your own pair → pass the gate → **publish your repository with its README**

Capstone (5', hard stop):

1. The **job** your workflow does
2. The **live run**
3. The **trust story**: one failure you caught + the check that caught it
4. One honest **number**
5. The **next step**

The trust story carries the most weight in grading. A documented, caught failure is stronger evidence than an unexamined success.

What you own now — public, under your name

One tool, on your GitHub:

- **Grounded prompts** that refuse rather than guess
- **A comparable-company valuation** of Apple, from live SEC data
- **Sanity checks and an evidence verifier** for everything a model writes
- **A screening workflow** on live filings, in LangGraph
- **A governed agent**, with every limit in code

A challenge for the coming week: choose one recurring task at your desk, build it as a workflow, and show a colleague.

Never ship a number you haven't verified.